

# Roteiro Prática 1

Samuel Tomazela

11 de setembro de 2026

## 1 Introdução

Relatório referente à Prática 1 da disciplina de Laboratório de Arquitetura e Organização de Computadores. Os códigos desenvolvidos estão detalhados a seguir, acompanhados de comentários explicativos. Todos os algoritmos foram testados com sucesso.

## 2 Códigos em Assembly MIPS

### 2.1 Exercício 1: Recursividade 1 [Portugol]

```
1
2 .data
3 firstMsg: .asciiz "Digite um numero: "
4 resultMsg: .asciiz "Novo valor: "
5 .text
6
7 addi $v0, $zero, 4 # L      string
8 la $a0, firstMsg # a0 contem endere  o de onde esta a mensagem
9 syscall
10
11 leInteiro:
12 addi $v0, $zero, 5 # L      inteiro
13 syscall
14
15 add $s0, $zero, $v0 #Guardando o valor de N em s0
16 addi $t0, $zero, 3
17 addi $s1, $zero, 1
18 addi $t2, $zero, 0 #Variavel que guarda a posicao do stackPointer
19
20 jal funcaoH
21 addi $v0, $zero, 4 # L      string
22 la $a0, resultMsg # a0 contem endere  o de onde esta a mensagem
23 syscall
24
25 add $a0, $s0, $zero
26 addi $v0, $zero, 1 # Imprime inteiro
27 syscall
28
29 addi $v0, $zero, 10
30 syscall
31
32 funcaoH:
33 sle $t1, $t0, $s0 # se 3 <= N t1 = 1 se nao (Sucesso) t1 = 0
34 beq $t1, $zero, return
35 beq $t1, $s1, case2
36 jr $ra
37 case2:
38 subi $sp, $sp, 4
```

```

39 sw $ra, 4($sp)
40 subi $sp, $sp, 4
41 sw $s0, 4($sp)
42 subi $s0, $s0, 2
43 jal funcaoH
44 add $a0, $zero, $s0 #salvando o resultado em $a0
45 subi $sp, $sp, 4
46 sw $a0, 4($sp)
47 lw $s0, 8($sp)
48 subi $s0, $s0, 3
49
50 jal funcaoH
51 add $a1, $s0, $s0
52 lw $s0, 8($sp)
53 lw $a0, 4($sp)
54 add $s0, $a0, $s0
55 add $s0, $a1, $s0
56 lw $ra, 12($sp)
57 addi $sp, $sp, 12
58 jr $ra
59
60
61 return:
62 add $s0, $s0, $s0
63 addi $s0, $s0, 1
64 jr $ra

```

## 2.2 Recursividade 2 [Portugol]

```

1  .data
2  firstMsg: .asciiz "Digite um numero: "
3  resultMsg: .asciiz "Novo valor: "
4  .text
5
6  addi $v0, $zero, 4 # L      string
7  la $a0, firstMsg # a0 contem endere  o de onde esta a mensagem
8  syscall
9
10 leInteiro:
11 addi $v0, $zero, 5 # L      inteiro
12 syscall
13
14 add $s0, $zero, $v0 #Guardando o valor de N em s0
15 addi $t0, $zero, 4
16 addi $s1, $zero, 1
17 addi $t2, $zero, 0 #Variavel que guarda a posicao do stackPointer
18
19 jal funcaoH
20 addi $v0, $zero, 4 # L      string
21 la $a0, resultMsg # a0 contem endere  o de onde esta a mensagem
22 syscall
23
24 add $a0, $s0, $zero
25 addi $v0, $zero, 1 # Imprime inteiro
26 syscall
27
28 addi $v0, $zero, 10
29 syscall
30
31 funcaoH:

```

```

32 sle $t1, $t0, $s0 # se 3 <= N t1 = 1 se nao (Sucesso) t1 = 0
33 beq $t1, $zero, return
34 beq $t1, $s1, case2
35 jr $ra
36 case2:
37 subi $sp, $sp, 4
38 sw $ra, 4($sp)
39 subi $s0, $s0, 4
40 jal funcaoH
41 add $a0, $s0, $s0 #salvando o resultado em $a0 * 2
42 addi $s0, $a0, 5 #s0 = a0 + 5
43 lw $ra, 4($sp)
44 addi $sp, $sp, 4
45 jr $ra
46
47
48 return:
49 add $t6, $s0, $s0
50 add $s0, $t6, $s0
51 jr $ra

```

## 2.3 Exercício 3: Máximo no Vetor

```

1  .data
2  vetorMsg: .asciiz "Digite o tamanho do vetor: "
3  firstMsg: .asciiz "Informe um numero: "
4  secondMsg: .asciiz "Maximo Valor do Vetor: "
5  vetor: .word 0:25
6
7  .text
8  addi $v0, $zero, 4 # L      string
9  la $a0, vetorMsg # a0 contem endere  o de onde esta a mensagem
10 syscall
11
12 addi $v0, $zero, 5
13 syscall
14
15 add $s0, $zero, $v0 #Guardando tamanho do vetor em s0
16
17 la $t0, vetor
18 add $t1, $zero, $zero #Variavel que vai andar pelo vetor
19 addi $t2, $zero, 4 #Variavel para andar pela memoria
20
21 loopLeitura:
22     beq $t1, $s0, fimLeitura
23     addi $v0, $zero, 4 # Le string
24     la $a0, firstMsg # a0 contem endere  o de onde esta a mensagem
25     syscall
26
27     addi $v0, $zero, 5 # Le inteiro
28     syscall
29
30     sw $v0, 0($t0)
31     add $t0, $t0, $t2
32     addi $t1, $t1, 1
33     j loopLeitura
34
35 fimLeitura:
36 #Prepara chamada de vetor
37 la $t0, vetor

```

```

38 add $t1, $zero, 1 #Variavel que vai andar pelo vetor
39 add $t3, $zero, $s0 #Variavel que vai guardar o tamanho do vetor
40 add $t4, $zero, $zero #maxAnterior
41 addi $s1, $zero, 1 #Guarda o valor 1 para compara o no beq
42 jal maxVetor
43
44 Final:
45     add $s2, $zero, $a0
46     addi $v0, $zero, 4 # Le string
47     la $a0, secondMsg # a0 contem endere o de onde esta a mensagem
48     syscall
49
50     add $a0, $zero, $s2
51     addi $v0, $zero, 1 # Imprime inteiro
52     syscall
53
54     addi $v0, $zero, 10 # Imprime inteiro
55     syscall
56 maxVetor:
57 beq $t3, $s1, caso1
58
59 subi $sp, $sp, 4 #Liberando espa o no stackPointer
60 sw $ra, 0($sp) #salvando ponto de volta
61
62 subi $sp, $sp, 4 #Liberando espa o no stackPointer
63 sw $t3, 0($sp) #salvando tamanho do vetor
64
65 subi $sp, $sp, 4 #Liberando espa o no stackPointer
66 sw $t0, 0($sp) #salvando posicao do vetor
67
68 add $t0, $t0, $t2
69 subi $t3, $t3, 1
70 jal maxVetor
71
72 add $t4, $a0, $zero #maxVetor = o valor retornado(a0)
73
74 lw $t0, 0($sp) #Recebendo posicao do vetor
75 addi $sp, $sp, 4 #Tirando espa o no stackPointer
76
77 lw $t3, 0($sp) #Recebendo posicao do vetor
78 addi $sp, $sp, 4 #Tirando espa o no stackPointer
79
80 lw $ra, 0($sp) #Recebendo posicao do vetor
81 addi $sp, $sp, 4 #Tirando espa o no stackPointer
82
83 lw $t5, 0($t0) #Recebendo valor do vetor $t5 = V[n]
84 sle $t6, $t4, $t5
85 beq $t6, $zero, caso2
86 beq $t6, $s1, caso1
87
88 caso1:
89 lw $a0, 0($t0)
90 jr $ra
91
92 caso2:
93 add $a0, $zero, $t4
94 jr $ra

```

## 2.4 Exercício 4: Máximo no Vetor

```

1  .data
2  firstMsg: .asciiz "Informe um numero, para imprimir todos os numeros menores
   at 1: "
3  space: .asciiz " "
4  .text
5
6  addi $v0, $v0, 4
7  la $a0, firstMsg
8  syscall
9
10 addi $v0, $zero, 5
11 syscall
12
13 add $s0, $v0, $zero
14 jal impOrdemDecrescente
15
16 #finaliza
17 addi $v0, $zero, 10
18 syscall
19
20 impOrdemDecrescente:
21 sle $t1, $s0, $t0
22 subi $sp, $sp, 4
23 sw $ra, 0($sp)
24 beq $t1, $zero, caso1
25 lw $ra, 0($sp)
26 addi $sp, $sp, 4
27 jr $ra
28
29 caso1:
30 add $a0, $s0, $zero
31 addi $v0, $zero, 1
32 syscall
33
34 addi $v0, $zero, 4
35 la $a0, space
36 syscall
37
38 subi $s0, $s0, 1
39 jal impOrdemDecrescente

```

## 2.5 Exercício 5: Fibonacci

```

1  .data
2  firstMsg: .asciiz "Informe um numero, para imprimir todos os numeros menores
   at 1: "
3  space: .asciiz " "
4  .text
5
6  addi $v0, $v0, 4
7  la $a0, firstMsg
8  syscall
9
10 addi $v0, $zero, 5
11 syscall
12
13 add $s0, $v0, $zero
14 jal impOrdemDecrescente
15
16 #finaliza

```

```

17 addi $v0, $zero, 10
18 syscall
19
20 impOrdemDecrescente:
21 sle $t1, $s0, $t0
22 subi $sp, $sp, 4
23 sw $ra, 0($sp)
24 beq $t1, $zero, caso1
25 lw $ra, 0($sp)
26 addi $sp, $sp, 4
27 jr $ra
28
29 caso1:
30 add $a0, $s0, $zero
31 addi $v0, $zero, 1
32 syscall
33
34 addi $v0, $zero, 4
35 la $a0, space
36 syscall
37
38 subi $s0, $s0, 1
39 jal impOrdemDecrescente

```

### 3 Comprovação - Resultados dos Testes

#### 3.1 Exercício 1: Recursividade 1

O primeiro exercício implementa uma função recursiva que avalia o valor de  $N$  e realiza chamadas empilhando o endereço de retorno ( $ra$ ) e parâmetros para calcular o resultado conforme a condição estabelecida.

A Figura apresenta a execução do algoritmo e a saída final obtida.

```
Digite um numero: 2
Novo valor: 5
-- program is finished running --

Digite um numero: 5
Novo valor: 23
-- program is finished running --

Digite um numero: 6
Novo valor: 37
-- program is finished running --

Digite um numero: 8
Novo valor: 91
-- program is finished running --
```

Figura 1: Resultado da execução do Exercício 1 (Recursividade 1).

### 3.2 Exercício 2: Recursividade 2

O segundo exercício aborda uma variação do cálculo recursivo, reduzindo o valor de entrada a cada chamada e manipulando a pilha (*sp*) para restaurar os registradores após a execução do caso base.

A Figura ilustra o resultado da execução no simulador.

```
Digite um numero: 3
Novo valor: 9
-- program is finished running --

Digite um numero: 7
Novo valor: 23
-- program is finished running --

Digite um numero: 15
Novo valor: 107
-- program is finished running --
```

Figura 2: Resultado da execução do Exercício 2 (Recursividade 2).

### 3.3 Exercício 3: Máximo no Vetor

O terceiro exercício realiza a leitura de um vetor de inteiros fornecido pelo usuário e utiliza uma função recursiva (`maxVetor`) para percorrer as posições da memória e determinar o maior valor armazenado.

A Figura demonstra a leitura dos dados e a exibição do elemento máximo do vetor.

```
Digite o tamanho do vetor: 10
Informe um numero: 1
Informe um numero: 2
Informe um numero: 5
Informe um numero: 45
Informe um numero: 67
Informe um numero: 92
Informe um numero: 36
Informe um numero: 0
Informe um numero: 124
Informe um numero: 24
Maximo Valor do Vetor: 124
-- program is finished running --
```

Figura 3: Resultado da execução do Exercício 3 (Máximo no Vetor).

### 3.4 Exercício 4: Impressão em Ordem Decrescente

O quarto exercício consiste na impressão regressiva dos números inteiros a partir do valor informado pelo usuário até a unidade, utilizando chamadas de procedimento para controlar a sequência.

A Figura apresenta a sequência gerada na saída padrão.

```
Informe um numero, para imprimir todos os numeros menores até 1: 23
23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
-- program is finished running (dropped off bottom) --
```

Figura 4: Resultado da execução do Exercício 4 (Ordem Decrescente).

### 3.5 Exercício 5: Fibonacci

O quinto exercício calcula o enésimo termo da sequência de Fibonacci utilizando a abordagem recursiva clássica com o gerenciamento dos registradores na pilha durante as chamadas duplas da função.

A Figura demonstra o valor final obtido para o número de elementos informado.

```
Sequencia de Fibonacci
Informe um Número: 10
A série de fibonacci para 10 elementos é:
55
-- program is finished running --
```

Figura 5: Resultado da execução do Exercício 5 (Fibonacci).